

Pimpri Chinchwad College of Engineering

Department of Information Technology

Cryptography and Network Security — Formative Assessment (FA-1 and FA-2)

Academic Year 2026-27

Secure Hospital Patient Record Exchange

Design and Implementation of a Cryptographically Protected Medical Report
Exchange System

Group Number: 3

PRN	Name of Student
123B1F040	Siddharth Kakade
123B1F061	Sharvani Marne
123B1F013	Harshvardhan Borude
123B1F011	Gaurav Biradar

Source Code: <https://github.com/Siddharthk17/Secure-Hospital>

Technology Stack: Angular 17 (TypeScript) frontend with Django 5 and Django REST Framework (Python) backend

1. Executive Summary

Hospitals routinely exchange sensitive patient reports between doctors and departments. During this exchange, attackers may intercept the reports and read them, or modify them silently. A changed dosage digit or a swapped allergy note is not only a privacy violation but a direct clinical danger.

This project designs, implements, and demonstrates a complete secure exchange system in which every patient report is protected by three cryptographic layers: (i) hybrid encryption using AES-256-GCM for the report body and RSA-2048-OAEP for delivery of the per-message data key, which guarantees that only the named recipient can read the report; (ii) SHA-256 integrity hashing with a second verification layer, so any modification is detected on opening; and (iii) RSA-2048-PSS digital signatures over a Hospital Certificate Authority public-key infrastructure, which proves authorship and prevents denial.

The system is a working full-stack application. The backend is built with Django and Django REST Framework in Python, with all cryptography isolated in a single audited module. The frontend is built with Angular and TypeScript and provides login, dashboard, report composition, inbox with triple verification display, patient registry, key management with rotation, a tamper-evident audit log viewer, a Security design page, and a controlled man-in-the-middle Attacker Laboratory that lets evaluators tamper with live reports and watch each attack get caught. Verification evidence includes five automated cryptographic unit tests (all passing), an offline attack transcript (5 of 5 behaviours as designed), an 11-step browser end-to-end suite (11 of 11 passing with zero console errors), and reproduction of every primitive in stock OpenSSL.

2. Problem Statement

Hospitals routinely exchange sensitive patient reports between doctors and departments. During this exchange, attackers may intercept or modify the reports, putting patient privacy and the reliability of medical data at risk. The objective is to develop a system that protects these reports during transmission by using encryption, hashing, and digital signatures to ensure confidentiality (only authorized recipients can read the report) and integrity (any modification of the report is detected).

Concretely, three failures must be prevented. First, eavesdropping: a passive attacker with tools such as Wireshark must learn nothing from captured traffic. Second, silent modification: an active attacker who flips, truncates, replays, or forges message content must be detected on every opening, with the report quarantined rather than displayed. Third, unattributable authorship: disputes about who sent a clinical order must be resolvable by cryptographic proof rather than by testimony.

3. Objectives

1. Identify the complete set of security requirements: confidentiality, integrity, authentication, non-repudiation, and availability.
2. Identify the realistic threats and vulnerabilities through an asset-first STRIDE threat model.
3. Select and justify standard cryptographic techniques for each requirement, with exact parameters and documented alternatives rejected.
4. Design a layered system architecture with block diagrams and numbered sender-to-recipient workflows.
5. Implement the design as a running full-stack application using Django (Python) for the backend and Angular (TypeScript) for the frontend.
6. Demonstrate the solution live, including simulated real attacks that are each caught and attributed to the layer that catches them.

4. Security Requirements

Requirement	Meaning in this system	Mechanism implemented	Where it is demonstrated
Confidentiality	Only the named recipient can read a report	AES-256-GCM bulk encryption; RSA-OAEP key wrap to one recipient; TLS in transit; role-based need-to-know; ciphertext-only storage	Report opens only for recipient; stolen-envelope attack returns UNREADABLE; outsider receives HTTP 403
Integrity	Any modification is detected	AES-GCM 128-bit authentication tag plus an independent SHA-256 recomputation; SHA-256 hash-chained audit log	Verification badges on every open; bit-flip attack fails the GCM tag; audit verify reports INTACT
Authentication	The sender is who they claim to be	RSA-PSS signatures; Hospital CA certificates binding identity to keys; JWT sessions; PBKDF2 passwords	CA VALID badge; Keys page fingerprints and certificates

Non-repudiation	The sender cannot later deny sending	Stored PSS signature over the digest plus hash-chained audit attribution	Verify-only endpoint re-proves authorship without decrypting
Availability	The service stays usable	Stateless API scalable behind a load balancer; request throttling; idempotent re-open; SQLite to Postgres upgrade path	Repeated opens succeed; throttles configured in settings

5. Threat Model

5.1 Assets and attackers

Assets, in order of clinical impact: (1) report plaintext, (2) decryption private keys, (3) signing private keys, (4) the CA signing key, (5) the audit log, (6) credentials and JWTs, (7) availability of the exchange itself.

Attackers assumed: A1, network eavesdropper; A2, active man-in-the-middle (read, flip, truncate, replay, inject); A3, forger without victim keys; A4, curious insider with a valid login but no need-to-know; A5, credential guesser; A6, log tamperer with database access; A7, thief of a private key or a stored envelope. Out of scope and stated openly: endpoint malware, physical workstation theft, and quantum attacks on RSA (the key-wrap layer is the designated post-quantum insertion point).

5.2 Trust boundaries

UNTRUSTED: network wire and browser JavaScript (holds no keys), stolen envelopes
-> API boundary: TLS, signed short-lived JWT, request throttling
-> TRUSTED SERVER: role checks on every object, audit appends, replay and context checks, single cryptographic module
-> SECRET: private keys and per-message data keys; database rows hold ciphertext only

Crossing each boundary requires something stronger: a TLS session, then a valid JWT, then a sender-or-recipient relationship, then private key material. Defence in depth means that even if TLS were defeated, the GCM tag and the PSS signature still stand; even with a valid login, role checks and OAEP key wrapping still stand.

5.3 STRIDE analysis (summary)

Spoofing: doctor impersonation is blocked by PSS signatures plus CA-bound keys (proven by the forged-signature attack failing); credential guessing is blocked by PBKDF2 with 600,000 iterations, login throttling, and 15-minute JWTs; fake-key substitution is blocked by CA certificates with visible fingerprints. Tampering: wire bit-flips fail the GCM tag with zero partial output; truncation and cut-and-paste fail tag, length, context binding, and digest comparison; stored hash or signature swaps fail recomputation and re-verification; audit edits break the hash chain at an exact sequence number; replays are rejected by the per-recipient nonce cache, context binding, and timestamp window. Repudiation fails because the stored signature and the chained seal entry re-prove authorship at any time. Information disclosure fails because the wire, the database, and a stolen envelope all expose only ciphertext and an OAEP-wrapped key that a single private key alone can open; direct URL access by an outsider returns HTTP 403 and is itself logged. Denial of service is contained by anonymous (30 per minute) and authenticated

(120 per minute) throttles over a stateless, horizontally scalable API. Elevation of privilege fails because JWTs are server-signed and every object access re-checks the sender, recipient, or administrator relationship, with violations logged as REPORT_ACCESS_DENIED.

6. Cryptographic Design and Justification

6.1 Requirement-to-mechanism traceability

Requirement	Choice	Reason in one line
Bulk confidentiality	AES-256-GCM (96-bit nonce, 128-bit tag)	One pass gives encryption and a tamper tag; hardware-accelerated; fails closed
Key delivery	RSA-2048-OAEP-SHA256 wrapping a fresh 256-bit DEK per message	RSA cannot bulk-encrypt, so it moves only the small key; fresh key per message bounds any leak to one report; OAEP has no padding-oracle history
Authorship	RSA-2048-PSS-SHA256 over the digest, separate signing keypair	Provably secure padding; constant-size signatures; key separation isolates compromise
Integrity anchor	SHA-256 over canonical JSON	One collision-resistant primitive anchors reports, key fingerprints, and the audit chain
Passwords	PBKDF2-HMAC-SHA256, 600,000 iterations	OWASP and NIST compliant work factor with per-user salts
Sessions	JWT, 15-minute access, 7-day rotating refresh with blacklist	Short replay window; stolen refresh tokens die on rotation
Freshness	CSPRNG nonce plus UUID message id plus signed timestamp	Three independent replay gates; timestamp signed inside the body
Audit	SHA-256 hash chain over sequence, actor, action, details, previous hash, time	Any edit breaks verification at an exact entry number

Key authenticity	Hospital CA over identity, fingerprint, and signing key	Recipients trust whose key it is, with rotation and revocation
------------------	---	--

6.2 The sealed envelope protocol (version 1)

Sealing (sender side): build deterministic canonical JSON of patient, subject, body, sender, recipient, timestamp, nonce, and message id; digest it with SHA-256; generate a fresh 256-bit data encryption key and 96-bit IV from the operating-system CSPRNG; encrypt with AES-256-GCM using additional authenticated data equal to sender, recipient, message id, and nonce; wrap the data key with the recipient's RSA-OAEP public key; sign the digest with the sender's RSA-PSS private key; store only ciphertext, wrapped key, IV, digest, and signature. Plaintext never touches the disk.

Opening (recipient side): unwrap the data key with the recipient private key (current key, then retired rotation-history keys); AES-GCM decrypt, which fails closed on any flipped or truncated bit; recompute SHA-256 and compare; verify the PSS signature with the sender public key; enforce nonce freshness, context binding, and timestamp window; re-validate the sender CA certificate; return plaintext only if every check passes, otherwise quarantine the report as TAMPER_ALERT with HTTP 409 and forensics.

6.3 Alternatives rejected and standards mapping

AES-CBC or CTR without a MAC was rejected (no integrity; padding oracles), as were RSA PKCS#1 v1.5 paddings (Bleichenbacher and forgery history), single keypairs for both uses (cross-protocol risk), MD5 and SHA-1 (publicly broken), DES, 3DES, and RSA-1024 (below the NIST 112-bit floor), static or reused IVs and data keys, unsalted fast password hashes, long-lived tokens, and any homebrew cipher (every primitive here reproduces in stock OpenSSL). The parameters satisfy NIST SP 800-38D (GCM), SP 800-56B and FIPS 186-5 (OAEP, PSS), SP 800-63B (passwords, throttling, short sessions), OWASP ASVS cryptographic controls, and the OWASP API Top 10 items for authorization, authentication, rate limiting, and auditing.

7. System Architecture

7.1 Context and components

```
Angular 17 + TypeScript frontend (JWT interceptor, route guards; holds no keys and performs no cryptography)
-- HTTPS with JWT Bearer tokens --
Django 5 + Django REST Framework API (stateless; all cryptography isolated in module crypto_service.py)
-- Object-Relational Mapping --
SQLite demonstration database, Postgres upgrade path (envelope rows plus hash-chained audit log; never plaintext bodies)
Hospital Certificate Authority (in-application signing key persisted with restricted permissions; issues identity certificates)
Actors: doctors, lab technicians, pharmacists, administrators (PBKDF2 passwords, 15-minute JWT sessions)
```

Component responsibilities. The module `crypto_service.py` performs key generation, fingerprinting, hybrid encryption and decryption, PSS signing and verification, envelope sealing and opening, and CA issue and verify operations; it is the only file that touches key material and the single boundary for future algorithm upgrades. The Report model persists the envelope and computes context binding; the Profile model holds role, department, dual keypairs, fingerprint, CA certificate, revocation flag, and the retired-key archive; `UsedNonce` enforces one-time nonces per recipient; `AuditLog` plus `audit.py` append and verify the hash chain. Views expose authentication, patients, sealing and sending, opening and verify-only checks, the six-mode Attacker Laboratory, key rotation, audit, and the security overview; every data access re-checks the sender, recipient, or administrator relationship. Angular pages provide login, dashboard, composition, inbox and sent lists, report detail with verification badges, patients, keys and PKI, audit with one-click verification, the Attacker Laboratory, and the Security design page.

Each user holds two RSA-2048 keypairs: an encryption pair (who may read for me) and a signing pair (proof it was me). Compromise of one use never defeats the other, and rotation and revocation can be scoped independently, following NIST and Trusted Computing Group practice.

7.2 Stored envelope anatomy

Field	Type	Purpose
id	UUID	Primary key and audit handle
patient, sender, recipient	Relations	Registry identity, authorship, and the sole decryption right
subject	Text up to 200 chars	Routing metadata

ciphertext_b64	Base64	AES-256-GCM output over the canonical plaintext with 128-bit tag
wrapped_dek_b64	Base64	RSA-OAEP-SHA256 of the 32-byte data key under the recipient encryption key
iv_b64	Base64	Fresh 96-bit CSPRNG nonce per message
digest_hex	64 hex chars	SHA-256 of the canonical plaintext; the integrity anchor
signature_b64	Base64	RSA-PSS-SHA256 of the digest under the sender signing key
nonce_hex, msg_id, sent_at	Replay context	One-time nonce, unique message id, signed timestamp
status	Enum	SEALED, DELIVERED, OPENED, or TAMPER_ALERT
sender_key_version, recipient_key_version	Integers	Which key generation sealed and opened the report

7.3 Principal workflows

Registration: administrator posts username, password, role, and department; the server validates, creates the Django user with PBKDF2 hashing, generates both RSA pairs from the OS CSPRNG, records the SHA-256 fingerprint, has the Hospital CA certify identity with fingerprints, stores the profile, and appends USER_REGISTER to the audit chain.

Sending: the server resolves patient and recipient (404 unknown, 409 revoked, 400 self-send), mints nonce and message id, canonicalises, hashes, encrypts with a fresh key and IV, wraps the key to the recipient, signs the digest, stores the envelope as DELIVERED, and appends REPORT_SEAL_SEND. Plaintext is discarded.

Opening: restricted to the recipient (the sender receives metadata with verify-only; anyone else receives HTTP 403 and a logged REPORT_ACCESS_DENIED). Decryption tries the current then retired keys, then the GCM tag, digest recomputation, PSS verification, replay checks, and CA re-validation run in order; success returns OPENED with plaintext and four badges, failure returns TAMPER_ALERT with HTTP 409 and forensics.

Rotation: one request archives the current encryption key, generates fresh pairs, re-issues fingerprint and certificate, and increments the key version, so history sealed under older keys

remains readable. Attacker Laboratory: six man-in-the-middle modes (bit-flip, truncation, signature forgery, hash swap, replay, theft by a keyless attacker) execute the genuine verification pipeline over tampered clones without ever mutating storage, reporting the verdict, the catching layer, and raw forensics.

7.4 Access control and deployment

Only the recipient row of the matrix permits decryption; notably, even the administrator cannot decrypt through the API, and database dumps contain no plaintext. Senders receive metadata with verify-only; outsiders receive HTTP 403 with logging; anonymous callers receive HTTP 401. Deployment places static Angular output and stateless Django workers behind a TLS-terminating reverse proxy with a shared database, encrypted backups, and scheduled audit-chain verification. All failure modes fail closed: tampering quarantines without partial output, wrong keys return unwrap errors with nothing disclosed, revoked recipients are refused at seal time, expired tokens demand refresh, and audit edits surface at an exact sequence number.

8. Implementation Summary

Backend (Python): Django 6.1.2, Django REST Framework 3.18.3, SimpleJWT 5.5.1 with 15-minute access and 7-day rotating blacklisted refresh, django-cors-headers, and the cryptography 50.0.2 package over OpenSSL. Configuration centralises the published crypto policy, throttles, and the production hardening switches that activate outside debug mode. A seed command creates five staff accounts with dual keypairs and certificates (dr_asha as sender, dr_rohan as recipient, lab_meera as outsider, pharma_kabir, admin) and three patients (MRN-1001 to MRN-1003).

Frontend (TypeScript): Angular 17 standalone components with typed services, a JWT HTTP interceptor, and an authentication route guard. Ten screens cover sign-in with demo credentials and login education, a dashboard with live counts and chain status, sealed sending with a visible staff fingerprint directory, inbox and sent lists with search, report detail with the four verification badges, patient registration, key management with rotation and staff creation, audit with one-click chain verification, the Attacker Laboratory with a 60-second demo script, and the Security design page with the block diagram. The bundled Sora variable font and glass styling keep the interface professional on desktop and mobile layouts.

Key endpoints: POST /api/auth/login and /api/auth/refresh; POST /api/auth/register; GET /api/me, /api/users, /api/users/pubkeys; POST /api/keys/rotate; GET and POST /api/patients; POST /api/reports/send; GET /api/reports/inbox, /sent, /search; GET /api/reports/open and /verify; POST /api/lab/attacker/; GET /api/audit and /api/audit/verify; GET /api/system/security-overview (open; drives the Security page so the interface cannot drift from backend truth).

9. Testing and Results

Automated cryptographic tests (python manage.py test): roundtrip seal-and-open with all three checks passing and byte-identical plaintext; single-bit ciphertext flip detected by GCM failure; forged signature from another key rejected; decryption with the wrong recipient key refused; CA certificate issuance and verification accepted. Result: 5 tests run, all passing.

Offline attack transcript (scripts/attacker_demo.py, no server required): honest open passes all checks with matching plaintext; bit-flip caught by tag failure; forged signature rejected; stolen envelope with the wrong key unreadable; CA certificate valid. Result: 5 of 5 behaviours as designed.

Browser end-to-end suite via Chrome DevTools automation (11 of 11 passing, zero console errors): login and dashboard with live policy; sealed send with digest; recipient open with four PASS badges and correct body; bit-flip, forgery, replay, and theft attacks each caught with the catching layer named; audit chain verified INTACT; key rotation from version 1 to 2 with continued readability; outsider access refused with HTTP 403; 390-pixel mobile layout verified;

Lighthouse scores of 97 accessibility, 100 best practices, and 90 search optimisation after fixing the two flagged items (page description and main landmark). Standards reproduction (scripts/openssl_interop.sh) confirms AES-GCM, RSA-OAEP, RSA-PSS, and SHA-256 in stock OpenSSL with no homebrew cryptography.

10. Demonstration Summary

The assessed demonstration follows an eight-minute script: frame the clinical stakes; send as dr_asha and read the digest and DELIVERED status; open as dr_rohan and read the four badges aloud; run the Attacker Laboratory through bit-flip, forgery, replay, and theft with attacker-then-catcher narration; verify the audit chain and prove outsider refusal as lab_meera; and close with keys, tests, standards, and roadmap. Recovery lines are scripted for every failure, including reading handled error messages as features and displaying forensics if any check ever misbehaves.

11. Limitations and Future Work

Stated openly: user private keys are server-custodied for the demonstration (production moves to hardware security modules with client-side end-to-end sealing, requiring no envelope-format change); the CA is a single file-persisted key rather than an offline root with intermediates and published revocation; stored mail has no forward secrecy if a recipient key is later stolen (mitigated by rotation, with periodic re-wrapping planned); TLS terminates at the reverse proxy with hardening switches ready; RSA-2048 is classical, with the wrap layer designated for hybrid post-quantum key encapsulation. The roadmap adds Postgres with encrypted backups, HSTS, WebCrypto sealing, Argon2id, department partitioning, audit shipping to a SIEM, and a formal retention and escrow policy.

12. Conclusion

The project delivers a complete, tested, and demonstrable secure hospital report exchange: hybrid encryption for confidentiality, double-layer hashing for integrity, signatures over a certificate authority for authentication and non-repudiation, and a stateless, throttled, audited service for availability. Every requirement maps to a mechanism, every mechanism to a visible proof, and every attack in scope to a catching layer the evaluators can trigger themselves. The system therefore answers the problem statement not with claims but with evidence: locked gibberish on the wire and in the database, four green badges on every honest opening, quarantine with forensics on every tampering, refusal with logging on every overreach, and an intact chained diary of everything that happened.

13. References

1. National Institute of Standards and Technology, SP 800-38D, Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM).
2. NIST SP 800-56B and FIPS 186-5, key establishment (RSA-OAEP) and digital signatures (RSA-PSS).
3. NIST SP 800-63B, Digital Identity Guidelines: authentication, memorised secrets, and session management.
4. OWASP Application Security Verification Standard 6.x, cryptographic and authentication controls; OWASP API Security Top 10 (authorization, authentication, rate limiting, auditing).
5. Django, Django REST Framework, and SimpleJWT documentation (authentication, throttling, token rotation).
6. Angular documentation (standalone components, HTTP interceptors, route guards).
7. Project source code: <https://github.com/Siddharthk17/Secure-Hospital> (backend, frontend, docs, scripts).